

Domain Analysis

1. Domain Definition and Stakeholder Analysis

Domain Definition

StoryFlowDSL targets the domain of interactive branching narratives, digital stories in which the player makes choices that alter the course of events, the state of the world, and the available endings. The domain is characterised by a graph of narrative scenes connected by guarded transitions, whose conditions depend on variables, inventory items, and prior choices.

The reference system used throughout this analysis is Life is Strange, a five-episode narrative adventure in which the protagonist Max Caulfield discovers a time-rewind power. Player decisions accumulate across episodes.. This butterfly-effect design makes the game an ideal, richly documented reference for modelling long-range conditional dependencies between scenes.

Scope

Inside the Domain	Outside the Domain
Scene graph (nodes + directed edges)	3-D world traversal and real-time movement
Player choices and dialogue trees	Physics engine and collision detection
State variables and boolean flags	Rendering pipeline, shaders, and visual assets
Inventory items and transitions	Audio/music system and voice acting
Conditions on transitions (required items, flag checks)	Online multiplayer or save-synchronisation services
Dice / probability rules on transitions	Platform-specific DRM or store APIs
Multiple endings reachable from the same start	Real-time object interaction
Cross-episode variable persistence	Animation state machines

Stakeholder Analysis

Stakeholder	Role and Interest
Game Designer	Primary user of the high-level DSL. Defines scenes, choices, conditions, and endings without programming. Needs expressive, domain-vocabulary notation.
Programmer	Consumes the pivot language or generated artifacts to implement or embed the story in a game engine.
QA / Playtester	Executes generated stories to verify reachability of all endings and absence of dead-end states.
Accessibility Reviewer	Checks that all narrative paths are explorable and that no ending is accidentally locked.
Tool Developer (MDE Engineer)	Designs and maintains the DSL, metamodel, transformations, and verification scripts.

2. Use Case Scenarios

UC1 — Conditional Scene Unlock (Investigation Gate)

Reference in Life is Strange

Max can only confront Juliet about the fake text message if she has already found the note in Dana's room (variable `found_note` = true). Without it, the confrontation choice is hidden from the dialogue menu.

Situation:

Max is in the dormitory corridor. The scene "Confront Juliet" has a single incoming transition guarded by `found_note == true`.

Configuration:

- Scene **corridor_dorm** has two outgoing choices: "Ignore the situation" (unconditional) and "Confront Juliet" (condition: `found_note == true`).
- Variable **found_note** is set to true in scene **dana_room** when the player examines the letter item.

Expected Behaviour:

- If `found_note == false`: the choice "Confront Juliet" is invisible or disabled.
- If `found_note == true`: both choices are available: "Ignore the situation" and "Confront Juliet".

UC2 — Butterfly Effect/Cross-Episode Variable Persistence

Reference in Life is Strange

Watering Lisa's plant in Episode 1 sets `plant_watered = true`. In Episode 2, the engine checks this flag to determine the plant's visual state.

Situation:

The story spans two chapters. An action in Chapter 1 sets a flag and in Chapter 2 reads that flag.

Configuration:

- Scene **dorm_room_ep1** contains an optional action "Water the plant". If the player chooses to water the plant then `plant_watered = true`.
- Scene **hallway_ep2** has two outgoing transitions to `alive_plant_scene` and `dead_plant_scene`, guarded by `plant_watered == true` and `plant_watered == false` respectively.

Expected Behaviour:

- If the player watered the plant, **hallway_ep2** routes to `alive_plant_scene`.
- Otherwise, `dead_plant_scene` is reached.

UC3 — Inventory-Gated Transition (Key and Lock)

Reference in Life is Strange

To enter the Dark Room in Episode 4, Max must first obtain Frank's RV key by either winning the negotiation peacefully or confronting him. The door transition is gated on `has_rv_key` in inventory.

Situation:

A locked door separates two areas. Max must possess the correct key item to be able to do the transition.

Configuration:

- Transition `enter_dark_room` has a condition: inventory contains `rv_key`.
- Item `rv_key` is added to inventory by scene **frank_negotiation** when the player selects the peaceful resolution.

Expected Behaviour:

- Without `rv_key`, the transition is blocked and the player must return to obtain the item.
- With `rv_key`, the transition fires and Max enters the dark room.

UC4 — Probabilistic / Dice-Rule Transition

Reference in Life is Strange

Some skill-check or tension scenes can be modelled as probabilistic outcomes.

Situation:

Max attempts a risky action. The outcome is determined by a dice roll with defined success probability.

Configuration:

- Transition `risky_jump` has a DiceRule: min 1, max 6 with a success threshold of 4 (success on 4-6).
- On success: scene **safe_landing**. On failure: scene **caught_by_guard**.

Expected Behaviour:

- The engine rolls a value in [1, 6].
- Value ≥ 4 leads to `safe_landing`. Value < 4 leads to **caught_by_guard**.

UC5 — Branching Endings (Sacrifice vs. Save)

Reference in Life is Strange

The final choice of the game, sacrifice Chloe to save Arcadia Bay, or sacrifice the town to save Chloe. This is a binary fork leading to two mutually exclusive ending scenes.

Situation:

The story is reaching an end and there's two endings available.

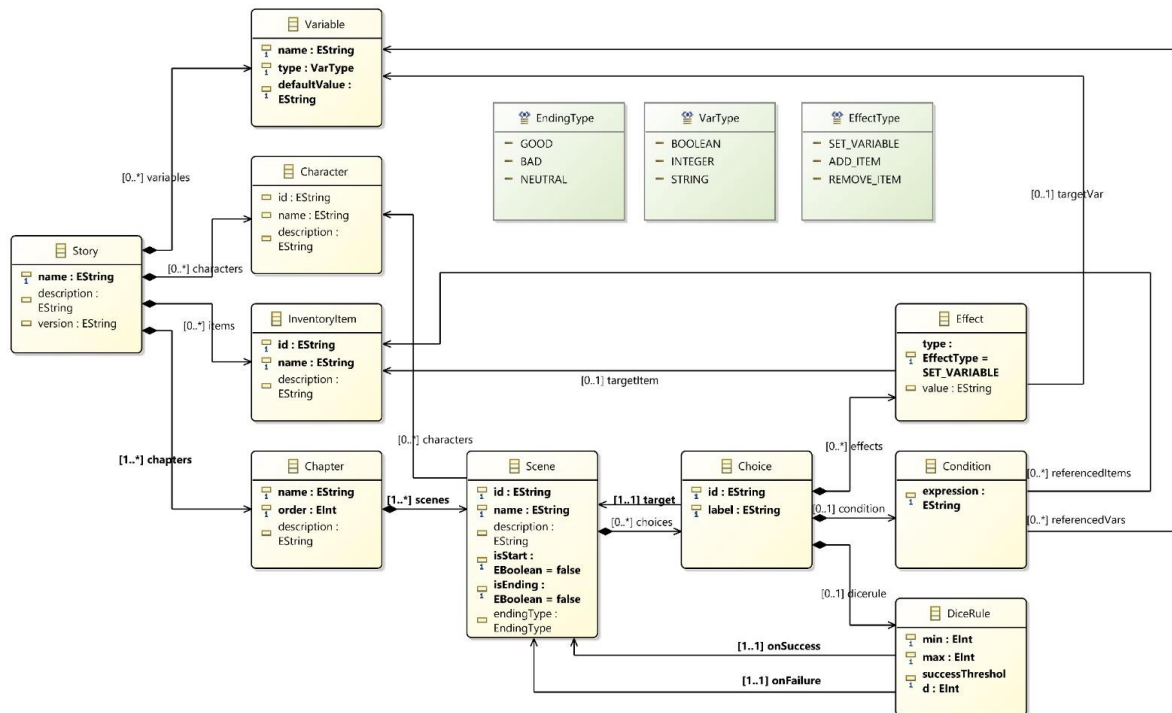
Configuration:

- Scene **lighthouse_decision** has two unconditional outgoing choices: "Sacrifice Chloe" -> **ending_sacrifice_chloe** and "Save Chloe" -> **ending_save_chloe**.
- Both ending scenes are marked `isEnding = true` with no outgoing transitions.

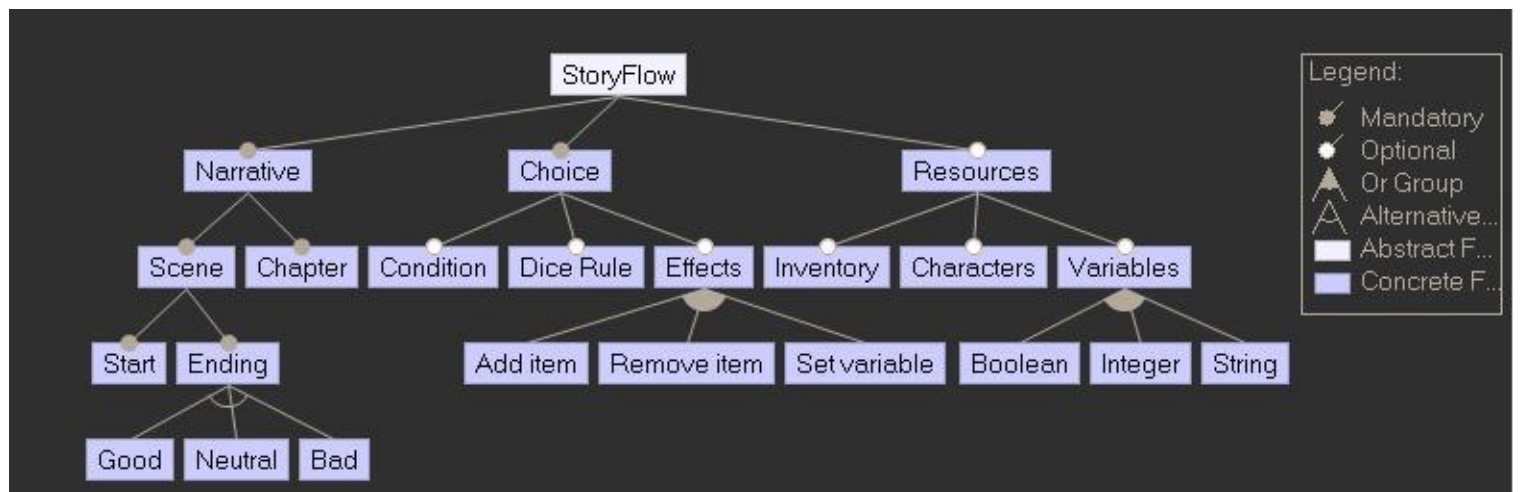
Expected Behaviour:

- Each choice leads to exactly one terminal scene.
- The formal verifier confirms both endings are reachable from the start scene and that no further transitions exist beyond them.

3. Domain Concept Model



4. Feature Model



5. Domain Constraints and Rules

1. Every story must contain exactly one scene with `isStart = true`.
2. No two scenes within the same story share the same id.
3. No two variables within the same story share the same name.
4. No two inventory items within the same story share the same id.
5. A scene marked `isEnding = true` must not have any outgoing choices.
6. A scene not marked `isEnding = true` must have at least one outgoing choice (to prevent dead-ends that are not declared endings).
7. Chapter order values must be strictly positive and unique.
8. Every choice must reference a scene id that exists within the same story.
9. Every variable referenced in a Condition or Effect must be declared in the story's variable list.
10. Every inventory item referenced in a Condition or Effect must be declared in the story's item list.
11. If a `DiceRule` is present on a choice, `min` must be strictly less than `max`, and `successThreshold` must be in the range `[min, max]`.
12. Every declared ending scene must be reachable from the start scene via at least one path.
13. Every non-start scene must be the target of at least one choice.
14. If a scene's outgoing choices are all guarded by conditions, those conditions must cover all possible variable states reachable at that point in the story.
15. The chapter order graph must be acyclic.

6. Functional Requirements

ID	Requirement
FR-01	The language must allow the author to declare a set of named scenes, exactly one of which is marked as the start scene. (UC1, UC2, UC5)
FR-02	The language must allow the author to define choices on scenes, with a human-readable label and a target scene reference. (UC1, UC5)
FR-03	The language must allow the author to attach a boolean condition to a choice, referencing declared story variables and inventory items. (UC1, UC2, UC3)
FR-04	The language must allow the author to declare story-scoped variables with a name, type (Boolean, Integer, String), and default value. (UC1, UC2)
FR-05	The language must allow the author to declare inventory items at story scope and express 'has item' conditions on choices. (UC3)
FR-06	The language must allow the author to attach state effects to choices: variable assignment (SET_VARIABLE) and item acquisition (ADD_ITEM). (UC2, UC3)
FR-07	The language must allow the author to attach a probabilistic dice rule to a choice, specifying a numeric range and a success threshold, with separate success and failure target scenes. (UC4)
FR-08	The language must allow the author to declare one or more terminal ending scenes, marked so that no outgoing choices are permitted on them, and guarantee that at least one ending scene is reachable from the start scene. (UC5)
FR-09	The language must allow the author to group scenes into ordered chapters or acts for structural organisation across long narratives. (UC2, UC5)

7. Non-Functional Requirements

ID	Requirement
NFR-01 (Usability)	The DSL syntax must be readable by non-programmer narrative authors: concept names must match domain vocabulary (Scene, Choice, Item, not Node, Edge, Token). A new user should be able to model a 5-scene story in under 30 minutes without reading more than the README.
NFR-02 (Validation)	Models that violate structural constraints (duplicate scene ids, missing start scene, invalid item references) must be rejected before any transformation or code generation is attempted, with clear, error messages.
NFR-03 (Formal Correctness)	The formal verification step must be capable of detecting at least: scenes from which no ending is reachable (dead-ends), endings that are permanently unreachable from the start, and any cycle from which no ending scene is reachable,
NFR-04 (Maintainability)	Adding a new type of transition effect (e.g. a new variable type or a new condition operator) must require changes only to the metamodel and EVL/ETL rules.
NFR-05 (Traceability)	Every generated artifact element (scene function in JS, state in NuSMV) must be traceable to its source scene/choice in the high-level model, either via naming convention or explicit trace links in the ETL transformation.
NFR-06 (Portability of Generated Artifacts)	The HTML/JS generated story runner must be self-contained in a single file and must execute without a local server in any modern browser, requiring no build step from the end user.